

Reviewer Pack — Continued-Fraction Depth of Greedy-Restriction Bias Sequence...

Entry cf05799b6ef2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 18:13:53 UTC

Continued-Fraction Depth of Greedy-Restriction Bias Sequence Bounds ACC⁰ Size

Entry ID: cf05799b6ef2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 18:13:53 UTC

1. Conjecture

Field A (mathematical branch): Diophantine approximation — continued fraction depth and Khinchin-Lévy statistics of rational approximants **Field B** (complexity object): ACC⁰ circuit size for explicit Boolean functions (Williams program; Sipser-style candidates)

Statement:

For a Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, define the *greedy-bias trajectory* $\tau(f)$ as the sequence of rationals p_i/q_i obtained by iteratively (for $i=1, \dots, t=\lceil \log_2 n \rceil$) extending the current partial restriction ρ_{i-1} by the (variable, value) pair (v,b) that maximizes $|\Pr_{x \sim \rho}[f(x)=1] - 1/2|$, with $p_i/q_i := \Pr[f=1 \mid \rho_i]$ expressed in lowest terms over denominators that are powers of two divided by the current restriction count. Let $D(f)$ be the total number of partial quotients across the continued-fraction expansions of $(p_1/q_1, \dots, p_t/q_t)$. Conjecture: for every f computable by an ACC⁰ circuit of size s and depth d , $D(f) \leq 8 \cdot d \cdot (\log_2 s)^2 + d \cdot \log_2 n$; equivalently, $D(f) > 8 \cdot d \cdot (\log_2 s)^2 + d \cdot \log_2 n$ on a single instance refutes the existence of such a circuit.

Rationale (proposer's reasoning):

ACC⁰ gates (AND, OR, MOD _{m} for small m) admit Hastad-style random-restriction collapse: greedily biased restrictions concentrate the conditional acceptance probability onto a small number of dyadic rationals whose continued-fraction expansions are short by elementary Diophantine arguments. The trajectory is *relational* (depends on the choice sequence of restrictions, not on the truth table alone via a single statistic), which lets it detect the modular-collapse structure of ACC⁰ that ring-only invariants miss. Sipser-style functions, engineered to remain width-robust against shallow restriction families, should force the bias trajectory through generically-irrational-looking approximants — long continued fractions are the obstruction signature.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 56e8bb0cfa08ebe8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds at $n \in \{12, 15, 18\}$: for ACC⁰ circuits (pop a) and PARITY (pop c), $D(f) \leq 8 \cdot d \cdot (\log_2 s)^2 + d \cdot \log_2 n$ on $\geq 80\%$ of trials (per (n,s,d) cell, median D under bound); for MAJORITY (pop d) and Sipser-hard (pop b), median D exceeds the bound on $\geq 50\%$ of $n=18$ trials. Any single ACC⁰/PARITY instance with $D > \text{bound}$ falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	UNCERTAIN	0.90	HITS	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - ACC⁰ lower bounds restriction method continued fraction - greedy restriction bias Boolean function circuit complexity rational approximation - Khinchin Levy partial quotients ACC circuit size explicit function

Top relevant hits considered: - [<http://arxiv.org/abs/1510.00068v2>] On the complete solution of the general quintic using Rogers-Ramanujan continued fraction - [<http://arxiv.org/abs/1111.6023v2>] On a General Sextic Equation Solved by the Rogers Ramanujan Continued Fraction - [<http://arxiv.org/abs/1908.05483v2>] Continued fractions over non-Euclidean imaginary quadratic rings - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/2407.04826v1>] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [<http://arxiv.org/abs/2003.09703v1>] Variance function of boolean additive convolution - [<http://arxiv.org/abs/1403.5456v1>] Levy processes with summable Levy measures, long time behavior - [<http://arxiv.org/abs/1405.4784v1>] An Explicit Formula For The Divisor Function

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def continued_fraction_length(p, q):
    if q == 0:
        return 0
    r = p % q
    return 1 + continued_fraction_length(q, r)
```

```

def build_random_circuit(n, size, depth):
    circuit = []
    gates = ['AND', 'OR', 'MOD_2', 'MOD_3', 'NOT']

    def add_gate(gate_type):
        if len(circuit) < size:
            input_indices = [random.randint(0, len(circuit) - 1) for _ in range(3)]
            circuit.append((gate_type, input_indices))

    for _ in range(depth):
        gate_type = random.choice(gates)
        add_gate(gate_type)

    return circuit

def evaluate_circuit(circuit, n):
    truth_table = [[0] * (2 ** n) for _ in range(2 ** n)]

    def eval_gate(gate, inputs):
        if gate[0] == 'AND':
            return all(truth_table[i][j] & truth_table[j][k] for i, j, k in zip(inputs, inputs[1:]))
        elif gate[0] == 'OR':
            return any(truth_table[i][j] | truth_table[j][k] for i, j, k in zip(inputs, inputs[1:]))
        elif gate[0] == 'MOD_2':
            return sum(truth_table[i][j] for i, j in zip(inputs, inputs[1:])) % 2
        elif gate[0] == 'MOD_3':
            return sum(truth_table[i][j] for i, j in zip(inputs, inputs[1:])) % 3
        elif gate[0] == 'NOT':
            return not truth_table[inputs[0]][inputs[1]]

    def eval_row(row):
        if len(row) == 1:
            return row[0]
        else:
            return eval_gate(circuit[row[-2]], [eval_row(row[:i]) for i in range(1, len(row))])

    for x in range(2 ** n):
        truth_table[x][x] = eval_row(list(range(n)))

    return truth_table

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [12, 15, 18]
    size_values = [20, 40, 80]
    depth_values = [2, 3, 4]

    metrics = []

    for n in n_values:
        for size in size_values:
            for depth in depth_values:
                circuit = build_random_circuit(n, size, depth)
                truth_table = evaluate_circuit(circuit, n)

```

```

p_i, q_i = 0.5, 1
trajectory = []

for _ in range(math.ceil(math.log2(n))):
    max_diff = -1
    best_var_val = None

    for var in range(n):
        for val in [0, 1]:
            new_p_i = sum(truth_table[i][var] * (val if i & (1 << var) else 1 - val))
            diff = abs(new_p_i - 0.5)
            if diff > max_diff:
                max_diff = diff
                best_var_val = (var, val)

    p_i, q_i = new_p_i, q_i * 2
    trajectory.append((p_i, q_i))

D_f = sum(continued_fraction_length(p, q) for p, q in trajectory)
metrics.append(D_f)

if not metrics:
    return {
        "metric_name": "D(f)",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

mean_D = sum(metrics) / len(metrics)
std_D = (sum((x - mean_D) ** 2 for x in metrics) / len(metrics)) ** 0.5
support_fraction = sum(1 for D in metrics if D <= 8 * depth_values[0] * (math.log2(size_values)))

return {
    "metric_name": "D(f)",
    "metric_value": mean_D,
    "instances_tested": len(metrics),
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": "" if support_fraction >= 0.8 else "mapping_undefined"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_D = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_D = (sum((r["metric_value"] - mean_D) ** 2 for r in results if r["metric_value"] is not None) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_D} std={std_D} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unsupported_operation")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_79724e62.py", line 132, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_79724e62.py", line 81, in run_trial
    circuit = build_random_circuit(n, size, depth)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_79724e62.py", line 40, in build_random_circuit
    add_gate(gate_type)

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_79724e62.py", line 35, in add_gate
    input_indices = [random.randint(0, len(circuit) - 1) for _ in range(3)]
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/usr/lib/python3.12/random.py", line 336, in randint
    return self.randrange(a, b+1)
           ^^^^^^^^^^^^^^^^^^^^^

```

```

File "/usr/lib/python3.12/random.py", line 319, in randrange
    raise ValueError(f"empty range in randrange({start}, {stop})")

```

ValueError: empty range in randrange(0, 0)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a ValueError in build_random_circuit (empty range in randrange) before any trials produced data, so neither support nor falsification conditions could be evaluated. | next: Fix build_random_circuit to seed the circuit with input literals before sampling gate inputs (ensure len(circuit) > 0 prior to randint), then rerun the 30-seed sweep across $n \in \{12, 15, 18\}$.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	369312
2	preregistration	claude_max	opus	0	0	9990
3	novelty	claude_max	opus	0	0	7397
4	novelty	claude_max	opus	0	0	7189
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18757
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	35058
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17387
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16258
9	judge	claude_max	opus	0	0	5713

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 487060 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/cf05799b6ef2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cf05799b6ef2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/cf05799b6ef2.tar.gz` (if generated)